# Vehicle 1 – Front Vehicle (Dashboard / Transmitter)

Original code supplied by the user. Replace the placeholder Blynk and Wi-Fi credentials before uploading.

```
#define BLYNK_TEMPLATE_ID   "YOUR_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "YOUR_TEMPLATE_NAME"
#define BLYNK_AUTH_TOKEN    "YOUR_NEW_AUTH_TOKEN"
#define BLYNK_PRINT Serial

#include <Arduino.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <esp_now.h>
#include <esp_wifi.h>

// -------- WIFI --------
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// -------- L298N MOTOR PINS --------
#define ENA 25
#define IN1 26
#define IN2 27
#define ENB 13
#define IN3 12
#define IN4 14

// -------- SPEED SENSOR PINS --------
#define RPM1_PIN 33
#define RPM2_PIN 32
const float PULSES_PER_REV = 1.0f;

// -------- BLYNK SPEED CONTROL --------
#define SPEED_VPIN V6
int speedPercent = 60;
int motorPWM = 153;

#define PWM_FREQUENCY 1000
#define PWM_RESOLUTION 8

// -------- ESP-NOW BROADCAST ADDRESS --------
uint8_t broadcastAddress[] = {
  0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF
};

// -------- COMMANDS --------
enum Command : uint8_t {
  STOP_CMD = 0,
  FORWARD_CMD,
  BACKWARD_CMD,
  LEFT_CMD,
  RIGHT_CMD
};

// -------- PACKET: MUST MATCH THE REAR VEHICLE --------
struct VehiclePacket {
  uint8_t command;        // 0 = STOP
  uint8_t speedPercent;   // 0 when stopped
  float rpm1;
  float rpm2;
  uint32_t sequence;
};

// -------- GLOBAL VARIABLES --------
volatile uint32_t pulses1 = 0;
volatile uint32_t pulses2 = 0;

float rpm1 = 0;
float rpm2 = 0;

Command commandCode = STOP_CMD;
String commandName = "STOP";

uint32_t lastRPMTime = 0;
uint32_t lastTelemetryTime = 0;
uint32_t sequenceNumber = 0;

bool pwmReady = false;
bool espNowReady = false;

// -------- INTERRUPTS --------
void IRAM_ATTR sensor1ISR() {
  pulses1++;
}

void IRAM_ATTR sensor2ISR() {
  pulses2++;
}

// -------- SPEED CONVERSION --------
int pwmFromPercent(int percent) {
  percent = constrain(percent, 0, 60);

  if (percent == 0) return 0;

  return (percent * 255 + 50) / 100;
}

// -------- MOTOR PWM --------
void writeMotorPWM() {
```

```
  motorPWM = pwmFromPercent(speedPercent);

  if (pwmReady) {
    ledcWrite(ENA, motorPWM);
    ledcWrite(ENB, motorPWM);
  }
}

// -------- STOP MOTORS --------
void stopMotors() {
  if (pwmReady) {
    ledcWrite(ENA, 0);
    ledcWrite(ENB, 0);
  }

  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

// -------- MOVEMENT FUNCTIONS --------
void setForward() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  writeMotorPWM();
}

void setBackward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  writeMotorPWM();
}

void setLeft() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
  writeMotorPWM();
}

void setRight() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
  writeMotorPWM();
}

// -------- APPLY COMMAND --------
void applyCommand() {
  // A zero speed setting must also transmit STOP.
  if (speedPercent == 0) {
    commandCode = STOP_CMD;
  }

  switch (commandCode) {
    case FORWARD_CMD:
      setForward();
      commandName = "FORWARD";
      break;

    case BACKWARD_CMD:
      setBackward();
      commandName = "BACKWARD";
      break;

    case LEFT_CMD:
      setLeft();
      commandName = "LEFT";
      break;

    case RIGHT_CMD:
      setRight();
      commandName = "RIGHT";
      break;

    default:
      stopMotors();
      commandName = "STOP";
      break;
  }

  Serial.printf(
    "Command: %s | Speed: %d%% | PWM: %d/255\n",
    commandName.c_str(),
    commandCode == STOP_CMD ? 0 : speedPercent,
    commandCode == STOP_CMD ? 0 : motorPWM
  );
}

// -------- BLYNK MOVEMENT BUTTONS --------
// V0 Forward, V1 Backward, V3 Left, V2 Right

BLYNK_WRITE(V0) {
  commandCode = (param.asInt() == 1) ? FORWARD_CMD : STOP_CMD;
  applyCommand();
```

```
}

BLYNK_WRITE(V1) {
  commandCode = (param.asInt() == 1) ? BACKWARD_CMD : STOP_CMD;
  applyCommand();
}

BLYNK_WRITE(V3) {
  commandCode = (param.asInt() == 1) ? LEFT_CMD : STOP_CMD;
  applyCommand();
}

BLYNK_WRITE(V2) {
  commandCode = (param.asInt() == 1) ? RIGHT_CMD : STOP_CMD;
  applyCommand();
}

// -------- BLYNK SPEED SLIDER --------
BLYNK_WRITE(V6) {
  speedPercent = constrain(param.asInt(), 0, 60);
  applyCommand();
}

// -------- ESP-NOW RECEIVE CALLBACK --------
// Optional: receives a matching packet from the rear vehicle.
void onDataReceived(
  const esp_now_recv_info_t *info,
  const uint8_t *data,
  int len
) {
  if (len != sizeof(VehiclePacket)) return;

  VehiclePacket packet;
  memcpy(&packet, data, sizeof(packet));

  Serial.printf(
    "Received rear packet: CMD=%u, SPEED=%u%%, RPM1=%.1f, RPM2=%.1f\n",
    packet.command,
    packet.speedPercent,
    packet.rpm1,
    packet.rpm2
  );

  if (Blynk.connected()) {
    String rearData =
      "CMD=" + String(packet.command) +
      ";SPEED=" + String(packet.speedPercent) + "%" +
      ";RPM1=" + String(packet.rpm1, 1) +
      ";RPM2=" + String(packet.rpm2, 1);

    Blynk.virtualWrite(V5, rearData);
  }
}

// -------- RPM CALCULATION --------
void updateRPM() {
  uint32_t now = millis();

  if (now - lastRPMTime < 1000) return;

  uint32_t elapsed = now - lastRPMTime;
  lastRPMTime = now;

  noInterrupts();
  uint32_t count1 = pulses1;
  uint32_t count2 = pulses2;
  pulses1 = 0;
  pulses2 = 0;
  interrupts();

  rpm1 = (count1 * 60000.0f) /
         (PULSES_PER_REV * elapsed);

  rpm2 = (count2 * 60000.0f) /
         (PULSES_PER_REV * elapsed);

  Serial.printf("RPM1: %.1f | RPM2: %.1f\n", rpm1, rpm2);
}

// -------- ESP-NOW TRANSMISSION --------
void sendTelemetry() {
  if (millis() - lastTelemetryTime < 200) return;

  lastTelemetryTime = millis();

  VehiclePacket packet = {};

  packet.command = (uint8_t)commandCode;
  packet.speedPercent =
      (commandCode == STOP_CMD) ? 0 : speedPercent;

  packet.rpm1 = rpm1;
  packet.rpm2 = rpm2;
  packet.sequence = sequenceNumber++;

  if (espNowReady) {
    esp_err_t result = esp_now_send(
      broadcastAddress,
      (const uint8_t*)&packet,
      sizeof(packet)
    );
    if (result != ESP_OK) {
```

```
      Serial.printf("ESP-NOW send error: %d\n", result);
    }
  }

  String dashboardData =
    "CMD=" + commandName +
    ";RPM1=" + String(rpm1, 1) +
    ";RPM2=" + String(rpm2, 1) +
    ";SPEED=" + String(packet.speedPercent) + "%";

  if (Blynk.connected()) {
    Blynk.virtualWrite(V4, dashboardData);
  }
}

// -------- SETUP --------
void setup() {
  Serial.begin(115200);

  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  pinMode(RPM1_PIN, INPUT_PULLUP);
  pinMode(RPM2_PIN, INPUT_PULLUP);

  stopMotors();

  bool pwmA = ledcAttach(ENA, PWM_FREQUENCY, PWM_RESOLUTION);
  bool pwmB = ledcAttach(ENB, PWM_FREQUENCY, PWM_RESOLUTION);

  pwmReady = pwmA && pwmB;

  if (!pwmReady) {
    Serial.println("PWM setup failed. Motors disabled.");

    while (true) {
      stopMotors();
      delay(1000);
    }
  }

  ledcWrite(ENA, 0);
  ledcWrite(ENB, 0);

  attachInterrupt(digitalPinToInterrupt(RPM1_PIN), sensor1ISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(RPM2_PIN), sensor2ISR, FALLING);

  // -------- WIFI --------
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Connecting to Wi-Fi...");
  uint32_t startTime = millis();

  while (
    WiFi.status() != WL_CONNECTED &&
    millis() - startTime < 20000
  ) {
    delay(250);
    Serial.print(".");
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED) {
    Serial.print("Wi-Fi connected. IP: ");
    Serial.println(WiFi.localIP());

    Serial.print("Wi-Fi channel: ");
    Serial.println(WiFi.channel());

    Blynk.config(BLYNK_AUTH_TOKEN);
    Blynk.connect(10000);

    if (Blynk.connected()) {
      Blynk.syncVirtual(V6);
    }

    // -------- ESP-NOW --------
    if (esp_now_init() == ESP_OK) {
      esp_now_register_recv_cb(onDataReceived);

      esp_now_peer_info_t peerInfo = {};
      memcpy(peerInfo.peer_addr, broadcastAddress, 6);
      peerInfo.channel = 0;
      peerInfo.encrypt = false;

      if (esp_now_add_peer(&peerInfo) == ESP_OK) {
        espNowReady = true;
        Serial.println("ESP-NOW broadcast ready");
      } else {
        Serial.println("Failed to add ESP-NOW peer");
      }
    } else {
      Serial.println("ESP-NOW initialization failed");
    }
  } else {
```

```
    Serial.println("Wi-Fi failed; Blynk and ESP-NOW unavailable.");
  }

  lastRPMTime = millis();
  Serial.println("Front vehicle ready.");
}

// -------- MAIN LOOP --------
void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    Blynk.run();
  }

  updateRPM();
  sendTelemetry();
}
```